

# Tema 26 Programación Modular, Diseño de Funciones, Recursividad, Librerías

---

## ÍNDICE

1. INTRODUCCIÓN
2. CONCEPTOS PREVIOS
3. PROGRAMACIÓN MODULAR
  - 3.1. PROGRAMA PRINCIPAL Y SUBPROGRAMAS
  - 3.2. ÁMBITO DE UN IDENTIFICADOR
4. DISEÑO DE FUNCIONES
  - 4.1. DECLARACIÓN
  - 4.2. INVOCACIÓN
  - 4.3. PASO DE PARÁMETROS
5. RECURSIVIDAD
  - 5.1. ESTRUCTURA
  - 5.2. CARACTERÍSTICAS
  - 5.3. TIPOS
6. LIBRERÍAS
  - 6.1. ESTRUCTURA
  - 6.2. CARACTERÍSTICAS
  - 6.3. TIPOS
7. CONCLUSIÓN
8. BIBLIOGRAFÍA

---

## 1. Introducción

- Paradigma de programación que consiste en la división del programa en varias partes que se comunican
- Importancia en el avance tecnológico.
- Fundamental para tecnologías modernas, dispositivos móviles y ordenadores actuales
- Forma parte del temario oficial de acceso a especialidades de Informática.
- Ubicado en el bloque de "Algoritmos y Programación".
- Importancia en el currículo educativo y tecnológico.

## 2. Conceptos Previos

- **Informática:** busca soluciones a problemas mediante algoritmos.
- **Programación:** traducción de algoritmos a un lenguaje entendible por la computadora.
- **Lenguaje de programación:** conjunto de reglas y símbolos para escribir programas.
- **Evolución:** de código "espagueti" a paradigmas estructurados y modulares.
- **Paradigmas:** Programación estructurada y modular
- Paradigma de programación que consiste en la división del programa en varias partes que se comunican

### 3. Programación Modular

- **Definición:** División de un programa en módulos independientes que puedan entenderse y manejarse de manera más fácil y que se comunican. Se denominan subprogramas o módulos y en el programa principal se llaman funciones.
- **Características:**
  - Trabajo en equipo.
  - Depuración.
  - Reutilización.
  - Mantenimiento.
  - Creación de librerías.
- **Objetivos:**
  - Simplificación del problema.
  - Independencia en programación.
  - Mejor legibilidad y mantenimiento.

#### 3.1 Programa Principal y Subprogramas

El programa principal se soluciona con el correspondiente programa o algoritmo principal y la solución de los subprogramas mediante funciones que podrán ser invocadas. La estructura del subprograma es similar a la de un programa y su función es resolver parte del programa principal de forma independiente.

- **Subprogramas internos:** Reside en el mismo archivo que el programa principal
- **Externos:** Se encuentra en otro archivo separado.

#### 3.2 Ámbito de un Identificador

- **Identificadores locales:** Definidos dentro de un subprograma, siendo su ámbito exclusivamente el de la función, no siendo accesible fuera de esta
- **Globales:** Definidos en el programa principal, siendo su ámbito todo el programa.

### 4. Diseño de Funciones

- **Definición:** Subprograma que recibe 0,1 o varios parámetros y devuelve un resultado asociado a la función, denominado retorno.
- **Tipos:** Propias del lenguaje vs definidas por el usuario. Las definidas por el usuario deben ser declaradas para ser invocadas.

#### 4.1 Declaración

- **Cabecera:** modificadores, tipo de retorno, nombre función y parámetros y tipo.
- **Cuerpo:** Instrucciones a realizar por la función

**Ejemplo: Función Java que calcula el cubo de un número**

```
public static long cubo(int n){           // Cabecera de la función
    return n*n*n                          // Cuerpo de la función.
}
```

La declaración puede variar de un lenguaje a otro.

### Ejemplo: Función Python que calcula el cubo de un número

```
def cubo(n):
    return n ** 3
```

## 4.2 Invocación

- Llamada con parámetros actuales.

## 4.3 Paso de Parámetros

Existe una correspondencia entre los parámetros formales, los que recibe la función, y los actuales, los que se envían a la función, por el cual los actuales sustituyen a los formales. Existen 2 métodos para establecer el paso de parámetros:

- **Paso por valor:** Los parámetros formales reciben una copia de los actuales.
- **Paso referencia:** Se envía la dirección de memoria del parámetro actual, siendo una variable compartida que puede ser modificada desde la función.

Los lenguajes modernos como Java, Python o Rust no implementan el paso por referencia, ya que gestionan la memoria de forma diferente.

# 5. Recursividad

- **Definición:** Herramienta de programación que reduce los algoritmos.
- Realiza llamadas a un método dentro del mismo método
- Simplifica el problema en otros más simples, denominados caso base.
- Función que se llama a sí misma.
- Muy útil en métodos que operan con estructuras de datos.

## 5.1 Estructura

- **Caso base:** Se resuelve sin recursividad, debe existir para salir de la recursividad.
- **Llamada recursiva:** La llamada recursiva debe progresar al caso base

### Ejemplo: Factorial Recursivo en Java

```
public static long factorial(int n) {
    // Caso base
    if (n == 0 || n == 1) {
```

```
        return 1;
    }
    // Llamada recursiva
    return n * factorial(n - 1);
}
```

## 5.2 Características

- Uso de memoria dinámica.
- Almacenamiento en pila.
- Profundidad de la recursividad el número de veces que la función se llama a si misma
- Muy potente en cálculo
- La función iterativa puede consumir menos recursos que la recursiva.

## 5.3 Tipos

- **Recursividad directa:** La función se llama a si misma.
- **Indirecta:** La función llama a otra función que a su vez vuelve a llamar a la primera función

## 6. Librerías

- **Definición:** Conjunto de implementaciones realizadas en algún lenguaje de programación para el desarrollo de funciones reutilizables.
- No pueden ejecutarse de modo aislado, deben ser invocadas para utilizarse en un programa. Algunas deben ser previamente importadas, ya que no forman parte de las librerías estándar del lenguaje. **Ejemplo:** Clase Scanner de Java

### 6.1 Estructura

- **Declaraciones globales.** Variables, constantes, funciones
- **Definición de funciones.** Desarrollo de las funciones de la librería
- **Llamadas a otras librerías.**

### 6.2 Características

- Ocultación de información.
- Reutilización del código
- Portabilidad de software
- Permite la invocación de cualquier función de la librería

### 6.3 Tipos

- **Librerías estándar:** Proporcionadas por el propio lenguaje, como la API de Java
- **Librerías de usuario.** Creadas por los usuarios para los desarrollos de sus implementaciones.

## 7. Conclusión

La programación modular representa un avance fundamental en la ingeniería del software, al permitir descomponer problemas complejos en subprogramas más pequeños y manejables. Como se ha visto, el

diseño de funciones y procedimientos, junto con un adecuado paso de parámetros y control del ámbito de los identificadores, facilita la creación de código legible, reutilizable y fácil de mantener.

La recursividad, aunque más compleja que la iteración tradicional, aporta una herramienta potente para resolver problemas de naturaleza recursiva, como recorridos de árboles o algoritmos de divide y vencerás, con una elegancia y claridad que a menudo supera a la solución iterativa.

Por su parte, las librerías constituyen el pilar de la reutilización en el desarrollo de software. Al encapsular funcionalidades en módulos independientes, permiten a los programadores construir sobre código ya probado, reduciendo errores y acelerando el desarrollo.

En conjunto, los conceptos tratados en este tema: modularidad, diseño de funciones, recursividad y librerías, son competencias esenciales para cualquier desarrollador, ya que sientan las bases del desarrollo de software estructurado y escalable, aplicable tanto en entornos académicos como profesionales.

## 8. Bibliografía

- Joyanes, L. (2020). Fundamentos de programación.
- Prieto, A. (2006). Introducción a la informática.
- López, L. (2004). Programación Estructurada.
- Hernández M. (2022). Estructuras de datos. Editorial Ra-Ma